

Index Choice for the Embeddings Table: Why HNSW, Not IVFFlat + REINDEX

Context

This project stores captured LLM traffic continuously and analyzes it in batch, later — it isn't a real-time filter deciding anything mid-request. That distinction mattered a lot for this decision, since it ruled out "the index must always be perfectly fresh" as a hard requirement and opened up scheduled-maintenance options that wouldn't make sense for a real-time system.

The problem

Started with pgvector's IVFFlat index on the prompt/completion embedding columns. IVFFlat's lists centroids are computed once, at CREATE INDEX time, based on whatever rows exist *at that instant* — they aren't recomputed automatically as new rows arrive. Our schema migration creates this index during initial setup, before the consumer has written a single row, meaning the index's clustering was built on an **empty table**. The centroids it's organized around are essentially meaningless, and nothing in the pipeline ever tells Postgres to redo that clustering as real data streams in afterward.

This stayed invisible during demo testing (make reset + full bulk load happens before anyone queries), but in a real deployment — consumer running continuously against live traffic — recall would quietly degrade over time with zero operational signal that anything was wrong.

Option 1: Scheduled REINDEX

Since this is a batch-analysis workload, not a real-time filter, a periodic rebuild seemed like the natural fit — no need for an always-fresh index if nothing queries it mid-request.

Rejected because: plain REINDEX takes a lock that blocks writes to the table for the entire duration of the rebuild. The Kafka consumer needs to keep inserting continuously even though analysis happens in batch — a nightly job that stalls ingestion isn't actually free, and would create a recurring window where the pipeline can't accept new events.

Option 2: REINDEX CONCURRENTLY

Avoids the write-blocking lock that plain REINDEX has.

Rejected because: it still rebuilds the *entire* index from scratch every single time it runs — not just the newly-arrived rows since the last rebuild. That cost scales with total table size, indefinitely. Trivial at today's ~10k rows; increasingly expensive as events_raw grows over months of continuous traffic. This solves the blocking problem but not the underlying "full rebuild, every time" cost, which only gets worse the longer the system runs.

Decision: HNSW

Sidesteps both problems structurally instead of working around them:

- **No lock-and-rebuild cycle** — new rows get inserted directly into the existing graph structure, no blocking, no scheduled maintenance window needed at all.
- **No cost that scales with total data size** — updates are incremental, so there's no "rebuild everything" bill that grows forever as the table grows.
- **Better recall-per-query-cost at scale** — generally outperforms a tuned IVFFlat on larger datasets in benchmarks, without needing to hand-tune lists/probes as data volume grows.

Given traffic volume here is expected to keep growing indefinitely rather than staying fixed, a fix whose cost doesn't scale with table size is the right one — not just the first one that came to mind.

Follow-up finding while making the switch

Even with HNSW, the default search-thoroughness settings are tuned for datasets much larger than ours — some queries were returning a decent-but-not-actually-closest match, silently, no error. Fixed with two changes: a higher-quality index construction setting, and more exhaustive search at query time. Because HNSW's approximate search has some inherent randomness, verified this by re-running a graduated test (exact match, near-duplicate paraphrase, injection-style query) multiple times with different queries, rather than trusting a single passing run.